

## TP 2

### Structure d'une page HTML5 sémantique

|                  |                                                             |
|------------------|-------------------------------------------------------------|
| Filière / Niveau | Licence Informatique — 1ère année                           |
| Module           | Développement Web                                           |
| Thème 1          | Intégration Web Responsive                                  |
| Durée            | 1 séance de 3 h, en binômes                                 |
| Prérequis        | TP1 (VS Code, Git, dépôt GitHub, cycle add → commit → push) |

### Objectifs du TP

À l'issue de ce TP, l'étudiant sera capable de :

- Écrire la structure de base d'un document HTML5 valide.
- Structurer une page avec les balises sémantiques (**header**, **nav**, **main**, **section**, **article**, **footer**).
- Insérer des liens, des images, des listes et des tableaux.
- Appliquer ces notions à la page d'accueil du projet fil rouge.

### I. Aperçu théorique

#### 1.1 Structure de base d'un document HTML

Un document HTML est un fichier texte composé de **balises** (`<p>...</p>`), qui peuvent porter des **attributs** (`lang="fr"`) et s'imbriquent les unes dans les autres.

- `<!DOCTYPE html>` : indique au navigateur qu'il s'agit de HTML5.
- `<html lang="fr">` : élément racine ; **lang** précise la langue de la page.
- `<head>` : informations sur la page, non affichées (`<meta charset="UTF-8">`, `<title>`, `<meta name="viewport" content="width=device-width, initial-scale=1">`).
- `<body>` : tout le contenu visible.

**NB** : `<meta charset="UTF-8">` : Indique l'encodage des caractères (accents, symboles, emojis). À placer en premier dans le head.

`<title>Club Informatique</title>` : Titre de la page, affiché dans l'onglet, les favoris et les résultats des moteurs de recherche. À ne pas confondre avec le h1, visible dans la page.

`<meta name="viewport" content="width=device-width, initial-scale=1">` : Fait suivre à la page la largeur réelle de l'écran, sans zoom initial. Indispensable pour le responsive.

**Règles à retenir** : Les deux balises `<meta>` sont **auto-fermantes** (pas de `</meta>`) : elles ne contiennent que des attributs.

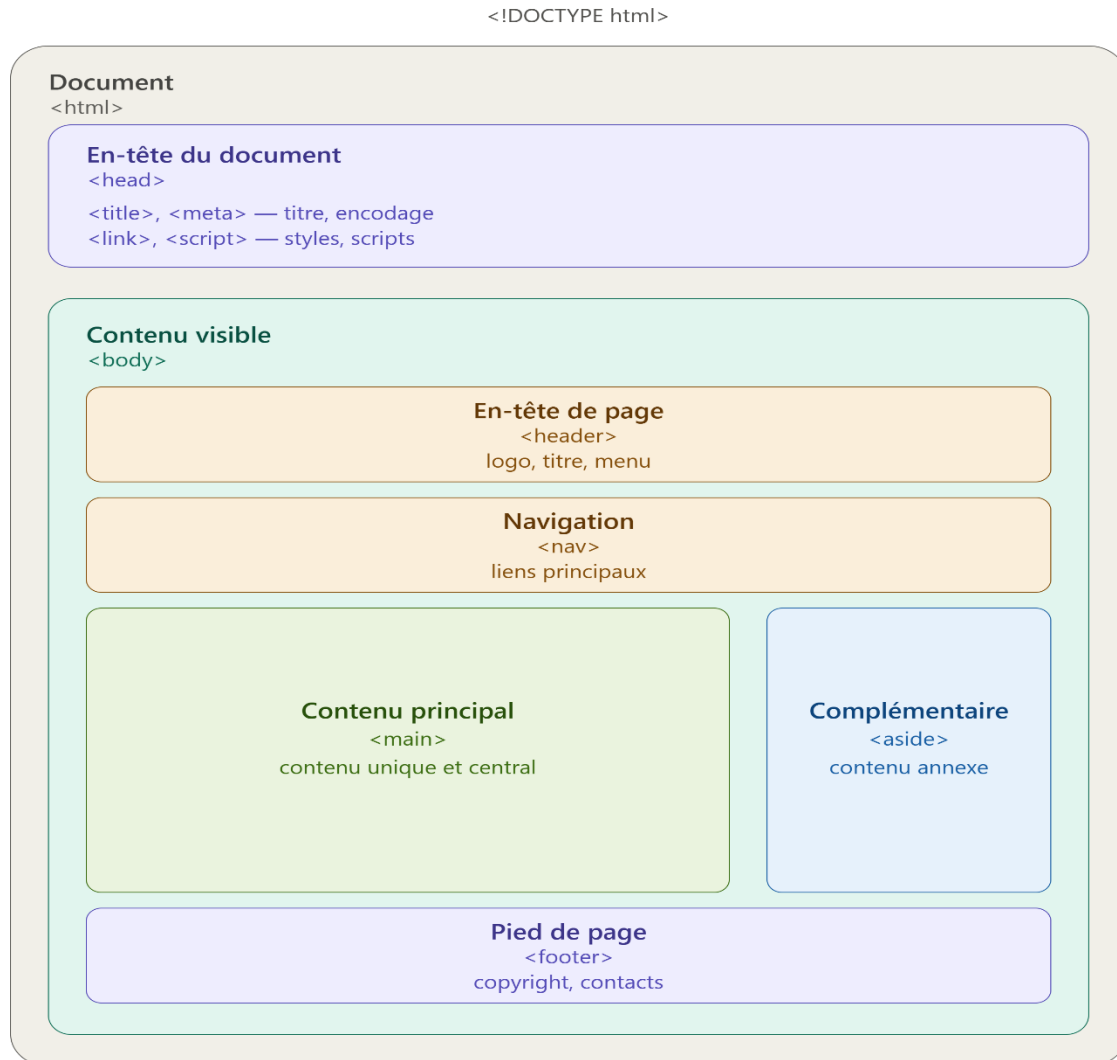
Fermer chaque balise, imbriquer proprement (la dernière ouverte est la première fermée), indenter le code, écrire les balises en minuscules.

## 1.2 Les balises sémantiques

Une balise **sémantique** décrit le *rôle* de son contenu, contrairement à `<div>` qui n'a aucun sens en soi. Elle améliore la lisibilité du code, l'accessibilité (lecteurs d'écran) et le référencement.

| Balise  | Rôle                                                                      |
|---------|---------------------------------------------------------------------------|
| header  | en-tête de la page ou d'une section (logo, titre)                         |
| nav     | bloc de navigation principal                                              |
| main    | contenu principal, <b>un seul</b> par page                                |
| section | regroupement thématique de contenu, avec un titre                         |
| article | contenu autonome, qui garde son sens isolé (actualité, fiche)             |
| aside   | contenu complémentaire ou note qu'on peut s'en passer                     |
| footer  | pied de page ou de section (les coordonnées de contact, le copyright,...) |

Les titres **h1** à **h6** structurent la hiérarchie : un seul **h1** par page, puis **h2**, **h3** sans sauter de niveau.



### 1.3 Liens, images, listes et tableaux

- **Lien** : `<a href="a-propos.html">À propos</a>`. Chemin *relatif* pour une page du site, URL complète pour un site externe.

**Un chemin relatif** indique où se trouve un fichier à **partir de l'endroit où l'on est**.

Avec cette arborescence :

```

TP2/
├── index.html
├── a-propos.html
├── images/
│   └── logo.png
├── pages/
│   └── contact.html

```

Depuis index.html, on écrit :

- | Pour atteindre...                   | Chemin relatif            |
|-------------------------------------|---------------------------|
| a-propos.html (même dossier)        | href="a-propos.html"      |
| logo.png (dans un sous-dossier)     | src="images/logo.png"     |
| contact.html (dans un sous-dossier) | href="pages/contact.html" |
- **Image** : ``. L'attribut `alt` est obligatoire (accessibilité, image non chargée).

- **Listes** : `<ul>` (non ordonnée) et `<ol>` (ordonnée), contenant des `<li>`.

#### Les balises

- ✓ `<ul>` (*unordered list*) : liste **non ordonnée**, l'ordre n'a pas d'importance. Le navigateur affiche des puces.
- ✓ `<ol>` (*ordered list*) : liste **ordonnée**, l'ordre compte. Le navigateur affiche des numéros (1, 2, 3...).
- ✓ `<li>` (*list item*) : **un élément** de la liste. Il se place obligatoirement à l'intérieur d'un `ul` ou d'un `ol`.

#### Exemple :

##### Code HTML

```
<h4>Matériel à apporter</h4>
<ul>
  <li>Un ordinateur portable</li>
  <li>Un chargeur</li>
  <li>Une clé USB</li>
</ul>

<h4>Déroulement de la séance</h4>
<ol>
  <li>Accueil et rappel de la séance précédente</li>
  <li>Exercices guidés</li>
  <li>Mini-projet en binôme</li>
</ol>
```

#### Résultat :

##### Matériel à apporter

- Un ordinateur portable
- Un chargeur
- Une clé USB

##### Déroulement de la séance

1. Accueil et rappel de la séance précédente
2. Exercices guidés
3. Mini-projet en binôme

- **Tableau** Un tableau présente des **données organisées en lignes et en colonnes**. Il s'écrit avec plusieurs balises imbriquées.

#### Les balises

| Balise                       | Rôle                                                           |
|------------------------------|----------------------------------------------------------------|
| <code>&lt;table&gt;</code>   | conteneur du tableau                                           |
| <code>&lt;caption&gt;</code> | titre du tableau (affiché au-dessus)                           |
| <code>&lt;thead&gt;</code>   | groupe l'en-tête (la ligne des titres de colonnes)             |
| <code>&lt;tbody&gt;</code>   | groupe les lignes de données                                   |
| <code>&lt;tr&gt;</code>      | une ligne (table row)                                          |
| <code>&lt;th&gt;</code>      | titre de colonne (table header), en gras et centrée par défaut |
| <code>&lt;td&gt;</code>      | cellule de données (table body)                                |

#### Exemple :

```
<table>
  <caption>Planning des activités</caption>
  <thead>
    <tr>
      <th>Jour</th>
      <th>Activité</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Mardi</td>
      <td>Atelier Python</td>
    </tr>
    <tr>
      <td>Vendredi</td>
      <td>Soirée jeux vidéo</td>
    </tr>
  </tbody>
</table>
```

#### Résultat réel dans le navigateur (sans CSS) :

| Planning des activités |                   |         |
|------------------------|-------------------|---------|
| Jour                   | Activité          | Salle   |
| Mardi                  | Atelier Python    | B12     |
| Vendredi               | Soirée jeux vidéo | Amphi 2 |

## II. Application pratique

Dans ce TP, le dossier et le dépôt s'appellent **TP2**. Il comprend deux exercices : l'exercice 1 porte sur la structure sémantique et les liens, l'exercice 2 sur les images, les listes et le tableau ; l'étape 3 applique cet ensemble à votre projet fil rouge (à réaliser lors d'une séance ultérieure, une fois les exercices 1 et 2 validés). Attention à la casse : **index.html** (tout en minuscules, sans espace ni accent dans les noms de fichiers).

### 2.1 Préparation

- Dans **TPWeb**, créer le sous-dossier **TP2**, puis dans **TP2** un sous-dossier **images** avec ces commandes :

```
cd ..  
mkdir TP2  
cd TP2  
mkdir images
```

- Configurer l'identité Git, **une seule fois pour toute l'année** (utiliser la même adresse e-mail que celle du compte GitHub) :

```
git config --global user.name "Votre Nom"  
git config --global user.email "votre.email@exemple.com"
```

- Initialiser le dépôt dans le dossier **TP2** :

```
git init  
git branch -M main
```

- Sur GitHub, créer un dépôt **vide** nommé **TP2** (sans **README**, **.gitignore** ni licence), puis relier le dossier à ce dépôt (à faire une seule fois) :

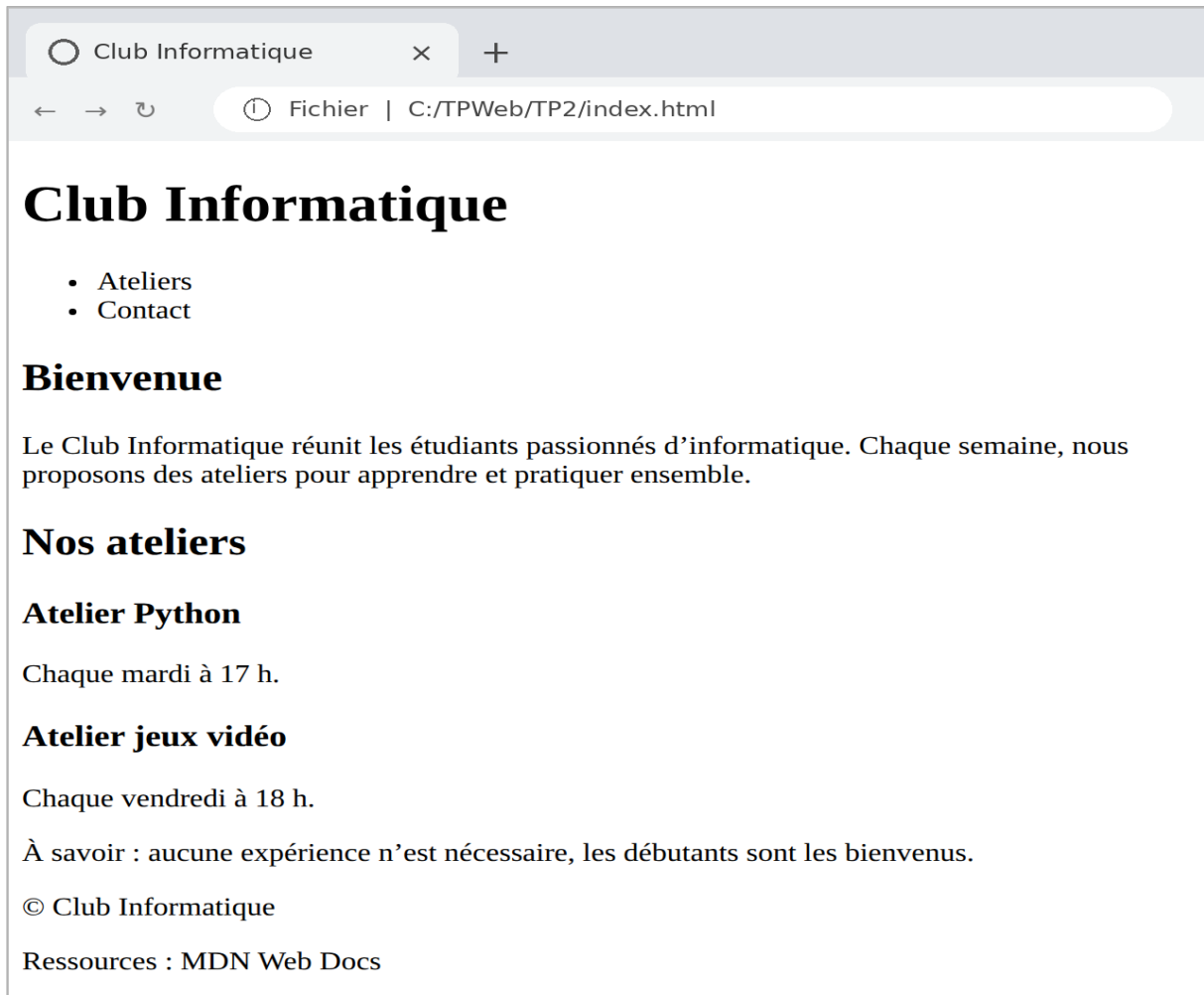
```
git remote add origin <URL_du_dépôt>
```

### 2.2 Exercice 1 : Page d'accueil du Club Informatique



#### 2.2.1 Structure de base et balises sémantiques

Réalisez la page Web dont l'aperçu est donné ci-dessous.



- Dans le dossier **TP2**, créez le fichier **index.html** avec le squelette complet (doctype, **lang**, charset, viewport). La page a comme titre « Club Informatique ».
- Organisez le contenu avec les balises sémantiques adaptées : **header**, **nav**, **main**, **section**, **article**, **aside**, **footer**.
- Le menu (**nav**) est une liste de deux éléments : *Ateliers* et *Contact*.
- Contraintes : un seul **h1** et un seul **main**, des titres sans saut de niveau, aucune **div**.  
(Cette contrainte vaut pour ce TP, afin de bien ancrer le réflexe sémantique ; la balise **div** reste un outil légitime dans d'autres contextes, notamment pour le style ou la mise en page.)

### 🔧 2.2.2 Aperçu de la page dans le navigateur avec Live Server

Pour visualiser la page web directement depuis VS Code, installer l'extension **Live Server** (par Ritwick Dey). Pour cela, ouvrir l'onglet **Extensions** (Ctrl + Shift + X), rechercher **Live Server**, puis l'installer.

Ensuite, faire un clic droit sur le fichier **index.html** et choisir « **Open with Live Server** ». Live Server lance un **serveur local** et ouvre automatiquement la page dans le navigateur par défaut. À chaque sauvegarde du fichier, la page est automatiquement actualisée dans le navigateur.

### 🚦 2.2.3 Cycle Git

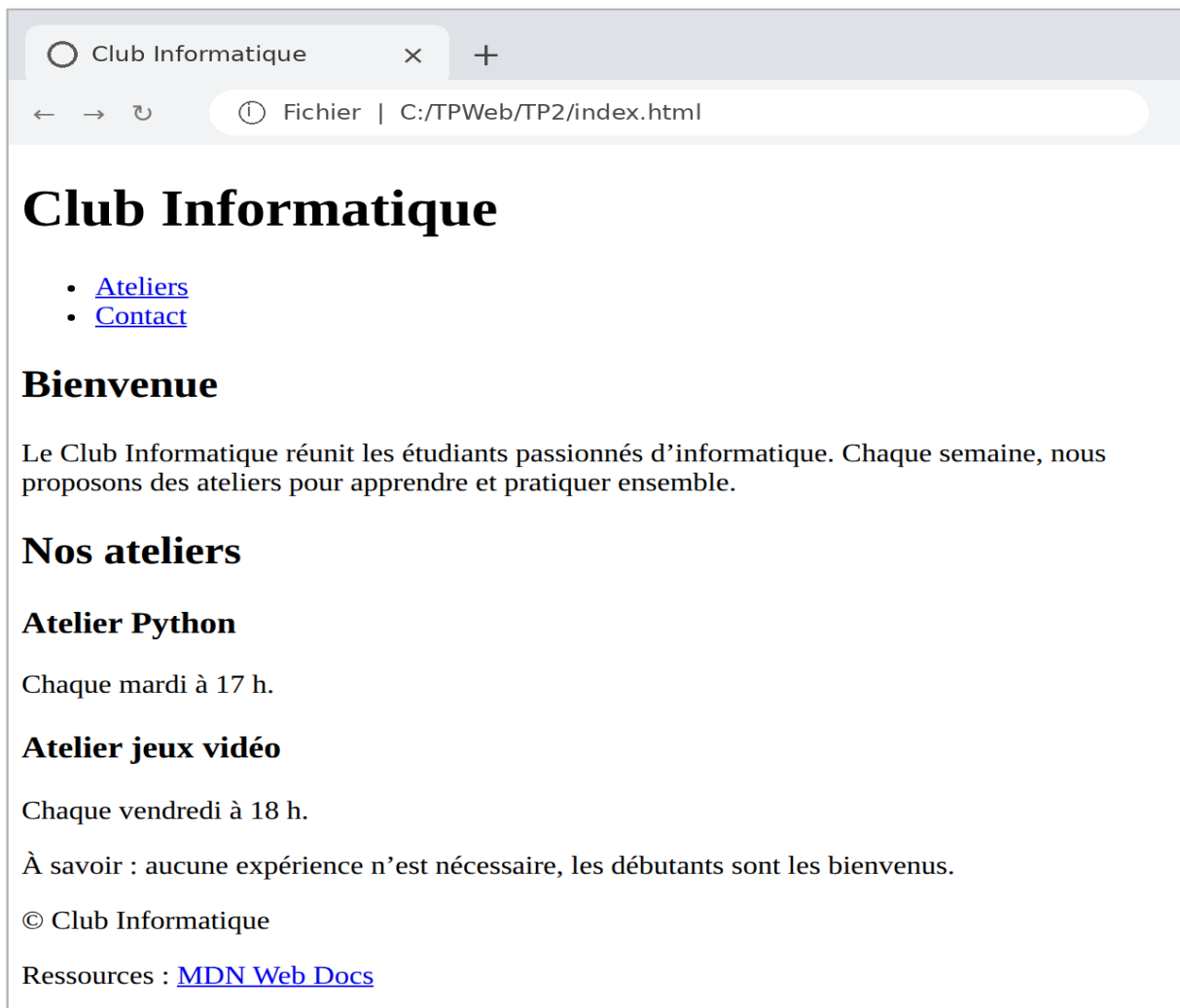
`git add .` → `git commit -m "Exercice 1 : structure de base"` → `git push -u origin main`.

#### Rappel :

- **push** → envoyer
- **origin** → le nom du dépôt distant (GitHub)
- **main** → la branche à envoyer
- **-u** → mémorise ce lien (à faire une seule fois)

### 🚦 2.2.4 Ajout des liens

Modifiez la page pour obtenir l'aperçu ci-dessous.



- Transformez les deux éléments du menu en liens : *Ateliers* vers [ateliers.html](#), *Contact* vers un lien [mailto: \(club@example.com\)](mailto:club@example.com).
- Dans le **footer**, transformez « MDN Web Docs » en lien vers <https://developer.mozilla.org/fr/>, ouvert dans un nouvel onglet (`target="_blank"`, `rel="noopener"`).
- Le lien *Ateliers* ne fonctionne pas encore : la page sera créée à l'Exercice 2.

### Vérification

Tester le lien *Contact* et le lien vers MDN, qui doit s'ouvrir dans un nouvel onglet ; avec **Live Server**

### 🚦 2.2.5 Cycle Git

git commit -m "Exercice 1 : ajout des liens ", puis git push

## 2.3 Exercice 2 : Page « Nos ateliers » (images, listes, tableau)

Réalisez la page Web **ateliers.html** dont l'aperçu est donné ci-dessous. Elle est reliée à la première par le lien *Ateliers*. Avant :

- Créez le sous-dossier **images** dans **TP2** et placez-y les images fournies (**logo.png** et **python.png**).
- Créez **ateliers.html** (titre : « Nos ateliers ») et reprenez la structure sémantique de l'Exercice 1.
- Le texte alternatif de **logo.png** est « Logo du Club Informatique », celui de **python.png** « Logo du langage Python ».
- Reproduisez les deux listes de l'atelier Python (matériel, déroulement) et le tableau de l'atelier jeux vidéo, avec son titre.

Ajoutez en bas de page deux liens : « Retour à l'accueil » vers **index.html**, et « S'inscrire » vers **inscription.html** (page à concevoir plus tard).



### Vérification

Tous les liens fonctionnent (dans les deux sens, depuis les deux pages), les images s'affichent, avec Live Server

### Cycle Git :

git commit -m "Exercice 2 : images, listes et tableaux ", puis git push